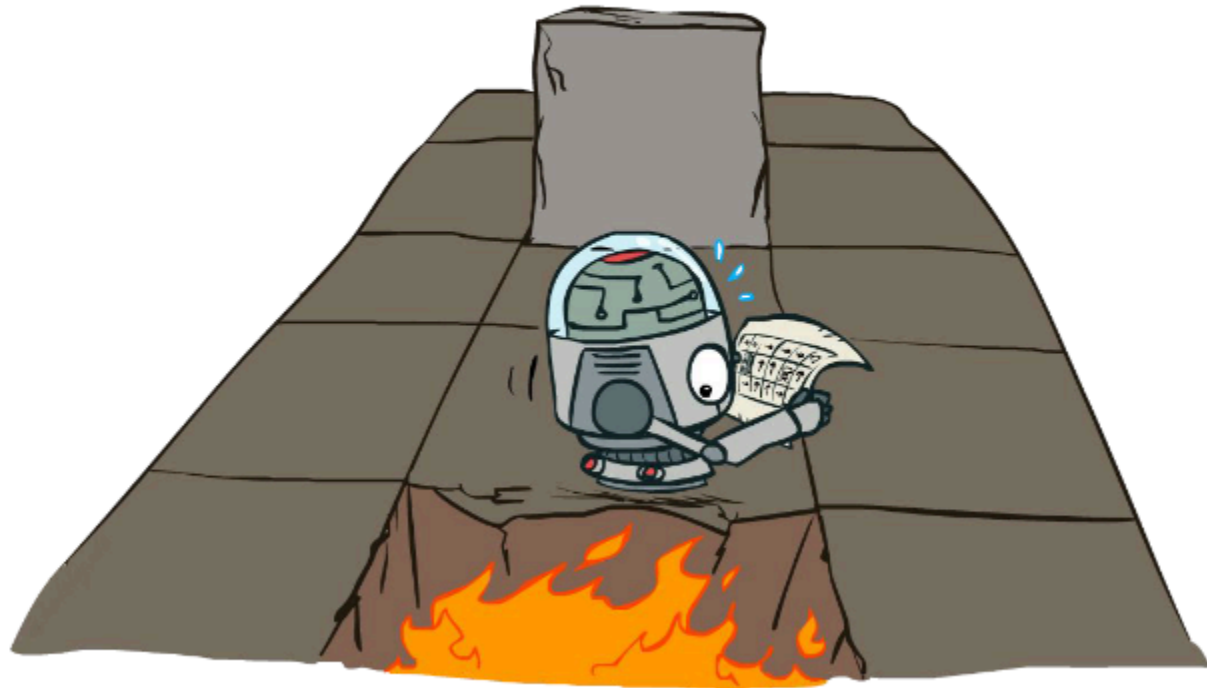


CS 188: Inteligência Artificial

Processos de Decisão de Markov II

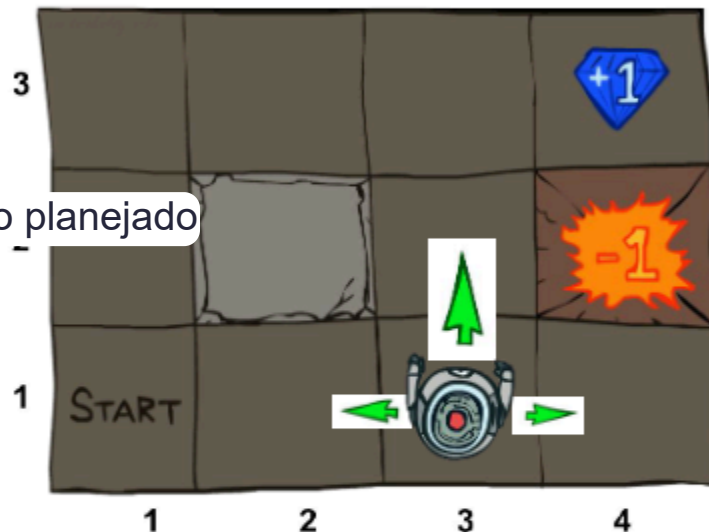


Instrutores: Emma Pierson e Peyrin Kao--- Universidade da Califórnia, Berkeley

[Esses slides foram criados por Dan Klein e Pieter Abbeel para o CS188 Introdução à IA na UC Berkeley. Todos os materiais do CS188 estão disponíveis em <http://ai.berkeley.edu>.]

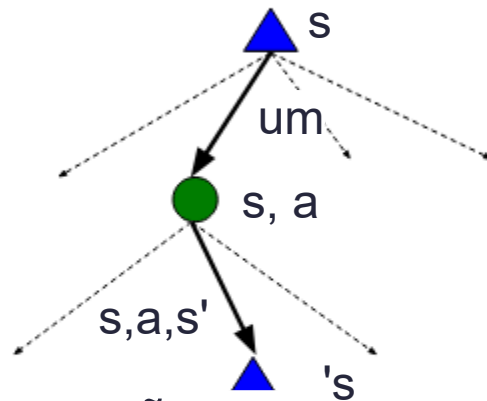
Resumo: Grid World

- Um problema labiríntico
 - O agente vive em uma grade
 - Paredes bloqueiam o caminho do agente
- Movimento barulhento: as ações nem sempre saem como planejado
 - 80% das vezes, a ação Norte leva o agente Norte
 - 10% das vezes, Norte leva o agente para Oeste; 10% Leste
 - Se houver um muro na direção em que o agente estaria Levado, o agente fica parado
- O agente recebe recompensas a cada passo de tempo
 - Pequenas recompensas "vivas" a cada etapa (podem ser negativas)
 - Grandes recompensas vêm no final (boas ou ruins)
- Meta: maximizar a soma das recompensas (descontadas)



Resumo: MDPs

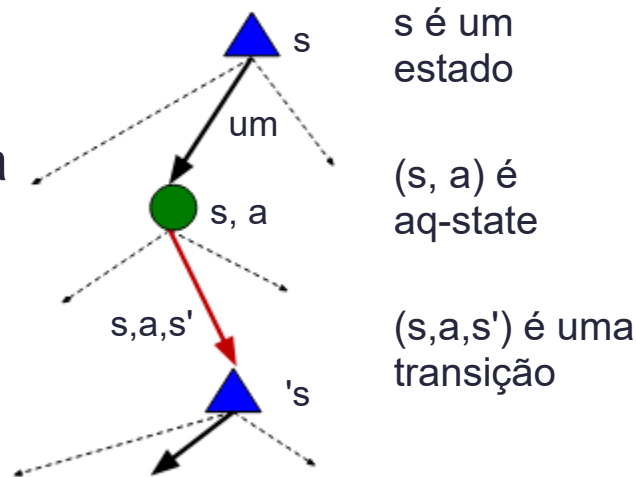
- Processos de decisão de Markov:
 - Estados S
 - Ações A
 - Transições $P(s'|s,a)$ (ou $T(s,a,s')$)
 - Recompensas $R(s,a,s')$ (e desconto γ)
 - Estado inicial s_0



- Quantidades:
 - Política = mapa de estados para ações
 - Utilidade = soma das recompensas descontadas
 - Valores = utilidade futura esperada de um estado (nó máximo)
 - Q-Valores = utilidade futura esperada de um q-estado (nó de chance)

Quantidades Ótimas

- Estrelas denotam otimalidade em todo o processo!
- O valor (utilidade) de um estado s : $V^*(s)$ = utilidade esperada começando em s e agindo de forma ótima
- O valor (utilidade) de um q-estado (s,a) : $Q^*(s,a)$ = utilidade esperada começando além de ter tomado a ação a a partir do estado s e (daí em seguida) atuando de forma ótima
- A política ótima: $\pi^*(s)$ = ação ótima a partir do estado s

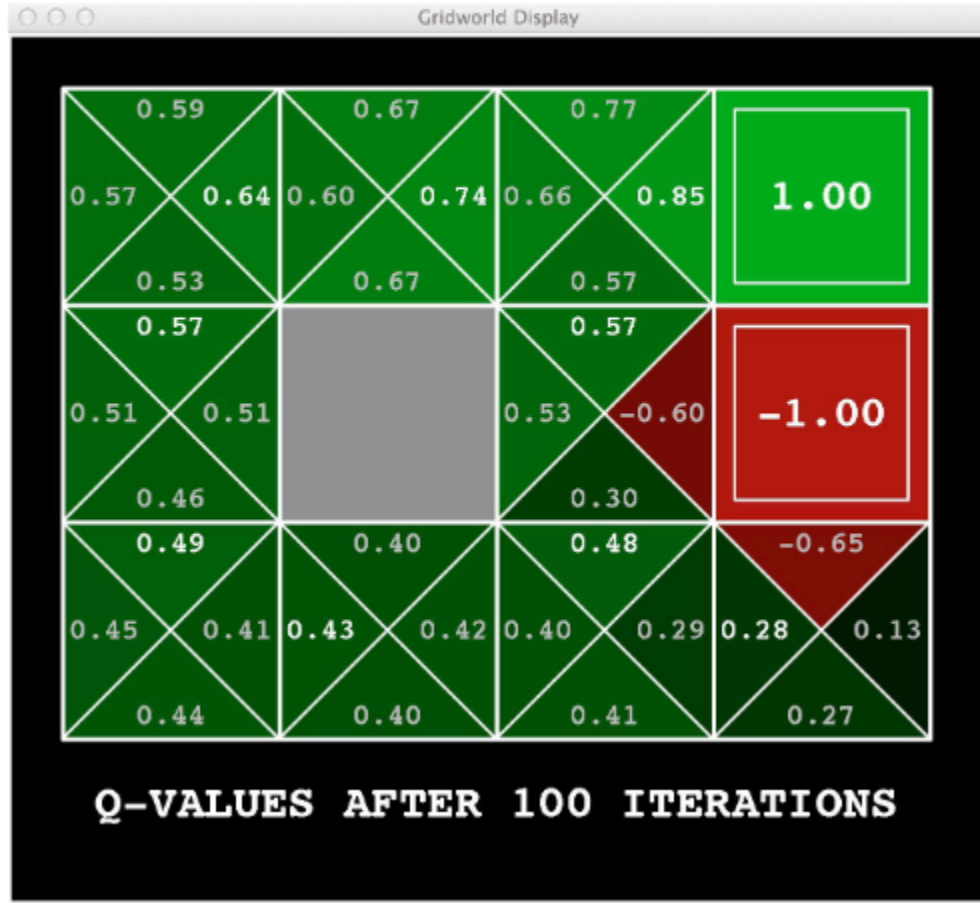


Valores do Gridworld V^*

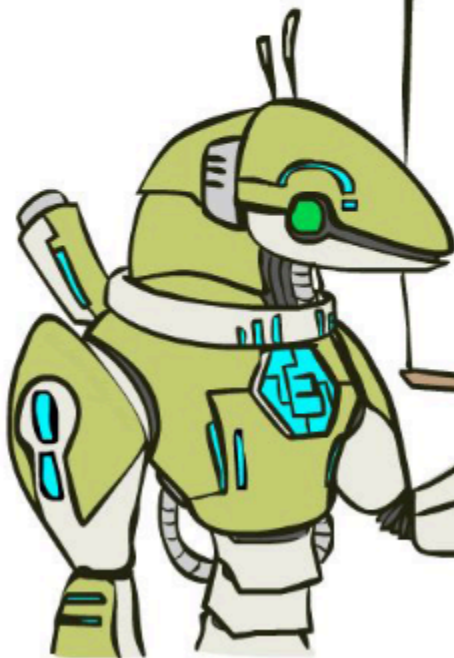


Desconto: 0,9
Ruído: 0,2
Recompensa de vida: 0

Gridworld: Q^*



As Equações de Bellman



Como ser ideal:

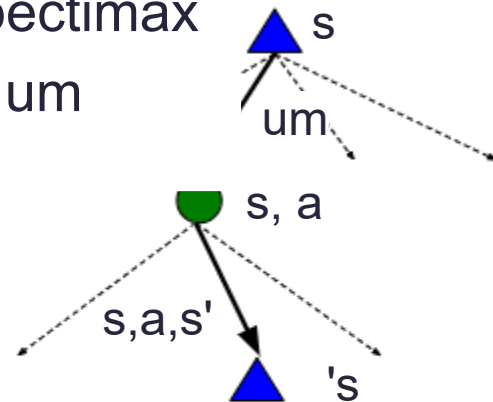
Passo 1: Tomar a primeira ação correta

Passo 2: Continuar sendo ótimo

As Equações de Bellman

- Definição de "utilidade ótima" via recorrência expectimax fornece uma relação de antecipação simples de um passo entre valores de utilidade ótimos

$$V^*(s) = \max_a Q^*(s, a)$$



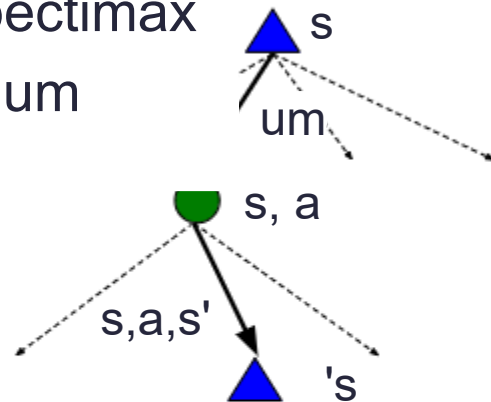
- Estas são as equações de Bellman, e elas caracterizam valores ótimos de uma forma que usaremos repetidamente

As Equações de Bellman

- Definição de "utilidade ótima" via recorrência expectimax fornece uma relação de antecipação simples de um passo entre valores de utilidade ótimos

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) =$$



- Estas são as equações de Bellman, e elas caracterizam valores ótimos de uma forma que usaremos repetidamente

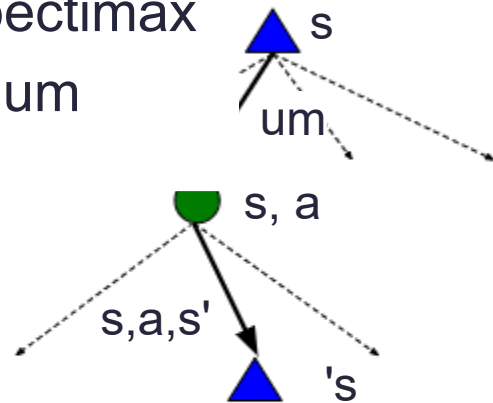
As Equações de Bellman

- Definição de "utilidade ótima" via recorrência expectimax fornece uma relação de antecipação simples de um passo entre valores de utilidade ótimos

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} \underbrace{T(s, a, s')}_{\text{Probabilidade de acabar no estado } s' \text{ Após a ação } A \text{ no estado } S}$$

Probabilidade de acabar no estado s'
Após a ação A no estado S



- Estas são as equações de Bellman, e elas caracterizam valores ótimos de uma forma que usaremos repetidamente

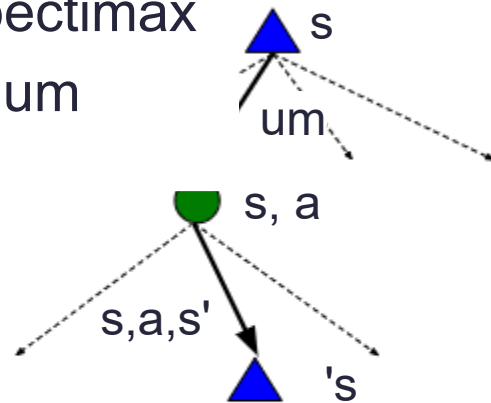
As Equações de Bellman

- Definição de "utilidade ótima" via recorrência expectimax fornece uma relação de antecipação simples de um passo entre valores de utilidade ótimos

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') \underbrace{\left[R(s, a, s') + \gamma V^*(s') \right]}$$

Utilidade de acabar no estado s
após tomar a ação a no estado s



- Estas são as equações de Bellman, e elas caracterizam valores ótimos de uma forma que usaremos repetidamente

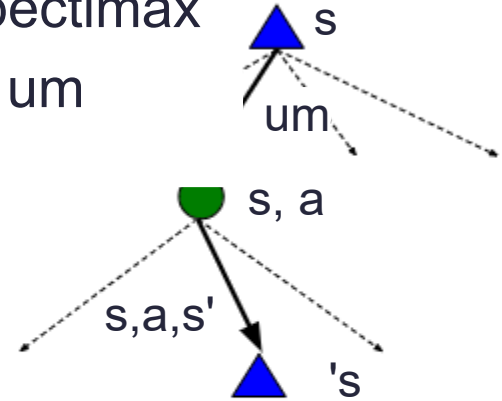
As Equações de Bellman

- Definição de "utilidade ótima" via recorrência expectimax fornece uma relação de antecipação simples de um passo entre valores de utilidade ótimos

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



- Estas são as equações de Bellman, e elas caracterizam valores ótimos de uma forma que usaremos repetidamente

Iteração de Valor

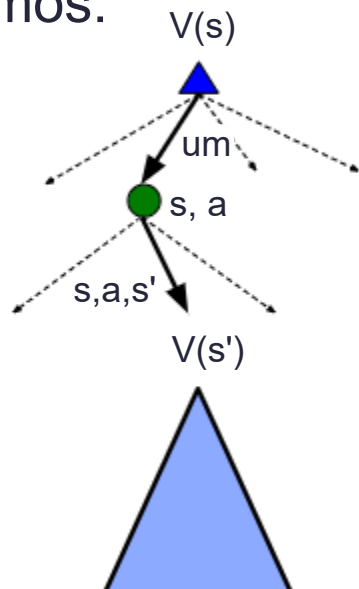
- As equações de Bellman caracterizam os valores ótimos:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- A iteração de valor os calcula:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Iteração de valor é apenas um método de solução por ponto fixo ... embora os vetores V_k também sejam interpretáveis como valores limitados no tempo



Valores com limite de tempo



Dia da Marmota, 1993

Valores com limite de tempo



Não Olhe para Cima, 2021

Valores com limite de tempo



Talvez Final Feliz, 2016

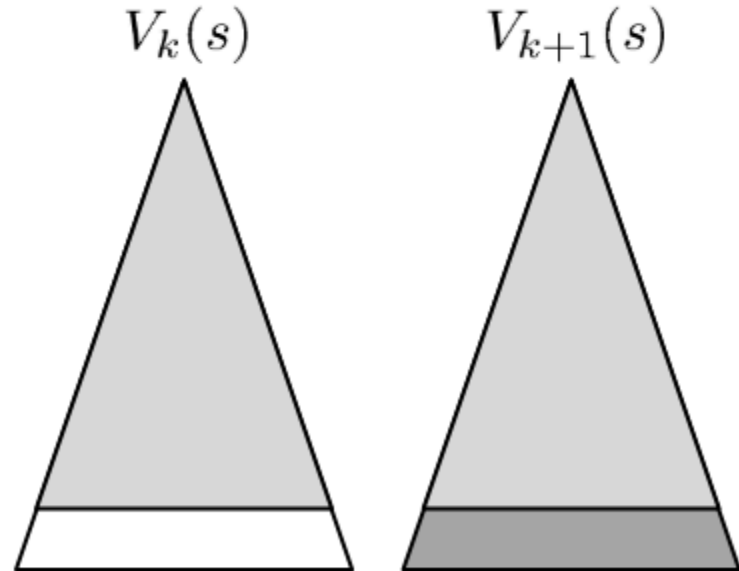
Convergência*

- Como sabemos que os vetores V_k vão convergir? ▪

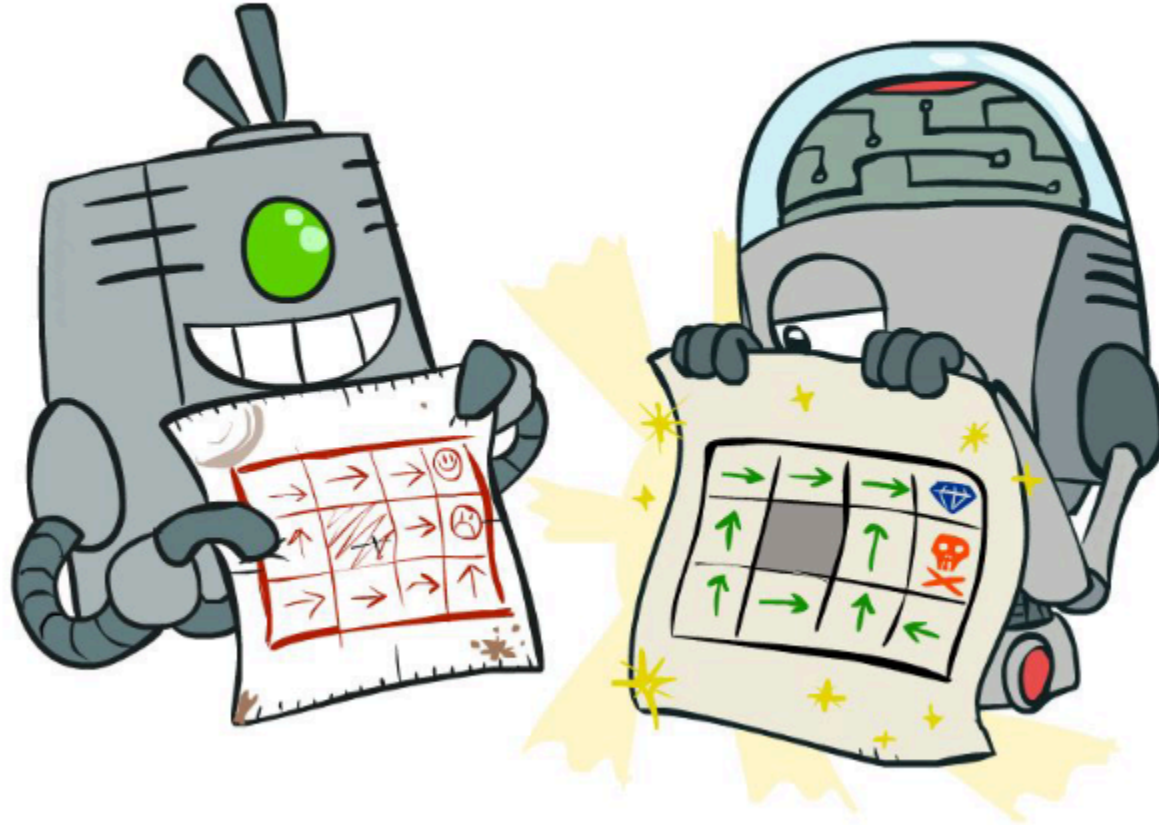
Caso 1: Se a árvore tem profundidade máxima M , então VM mantém os valores ▪ reais não arrunados

Caso 2: Se o desconto for menor que 1

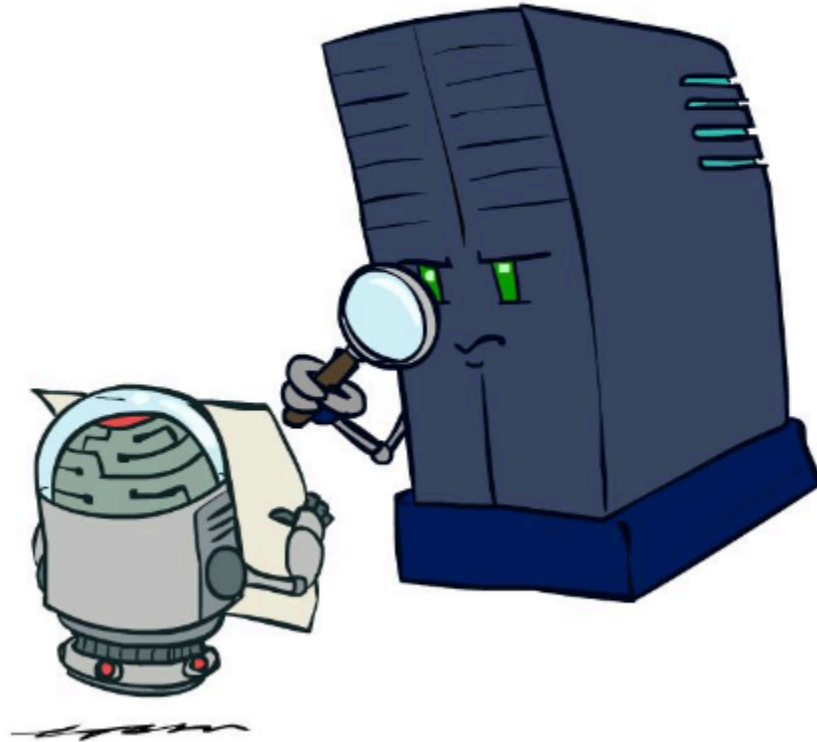
- Esboço: Para qualquer estado, V_k e V_{k+1} podem ser vistos como profundidade+1 expectimax resulta em árvores ▪ de busca quase idênticas. A diferença é que na camada inferior, V_{k+1} tem recompensas reais enquanto V_k tem zeros ▪. Essa última camada é, no máximo, toda RMAX ▪. No pior dos casos, RMIN ▪. Mas tudo é descontado em γ_k até esse ponto ▪. Então V_k e V_{k+1} são no máximo $\gamma_k \max |R|$ diferente ▪. Então, à medida que k aumenta, os valores convergem



Métodos de Políticas

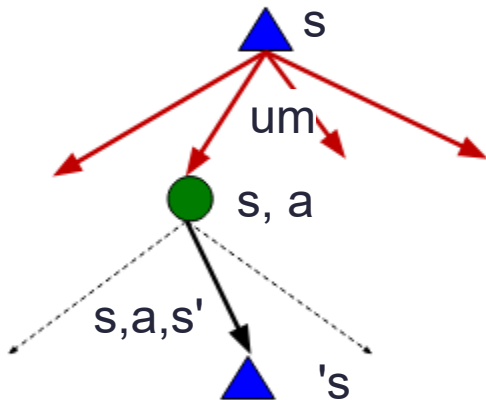


Avaliação de Políticas

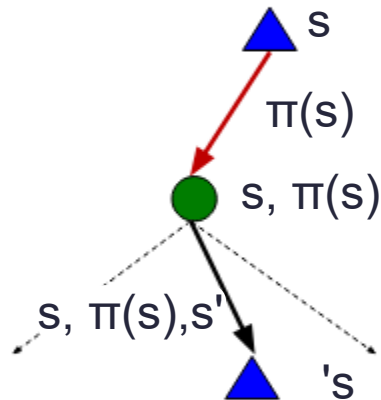


Políticas Fixas

Faça a ação ideal



Faça o que π mandar fazer



- Árvores expectimax maximizam todas as ações para calcular os valores-ótimos. Se fixássemos alguma política $\pi(s)$, a árvore seria mais simples – apenas uma ação por estado
 - ... embora o valor da árvore dependesse de qual política fixássemos

Utilidades para uma Apólice Fixa

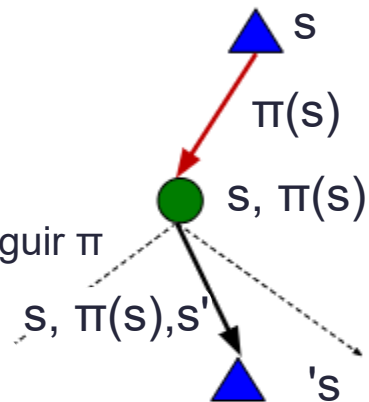
- Outra operação básica: calcular a utilidade de um estado s sob uma política fixa (geralmente não ótima)

- Defina a utilidade de um estado s , sob uma política fixa π :

$V^\pi(s)$ = recompensas totais descontadas esperadas a partir de s e a seguir π

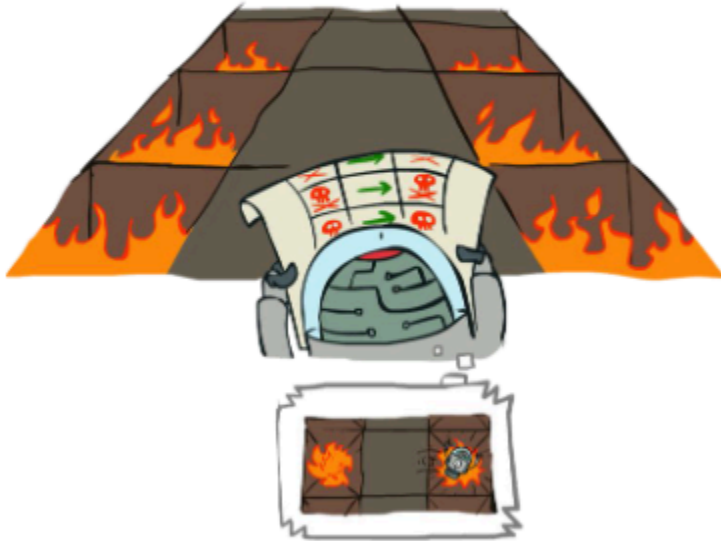
- Relação recursiva (visão antecipada de um passo / equação de Bellman):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$



Exemplo: Avaliação de Políticas

Sempre Vá Certo



Sempre Siga em Frente



Exemplo: Avaliação de Políticas

Sempre Vá Certo

-10.00	100.00	-10.00
-10.00	1.09 ▶	-10.00
-10.00	-7.88 ▶	-10.00
-10.00	-8.69 ▶	-10.00

Sempre Siga em Frente

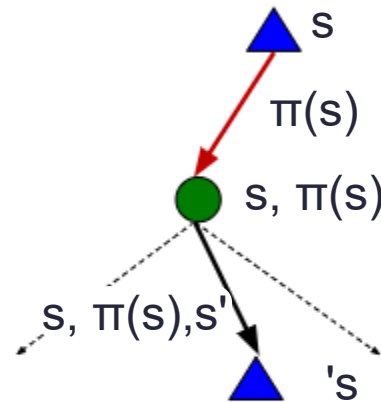
-10.00	100.00	-10.00
-10.00	70.20 ▲	-10.00
-10.00	48.74 ▲	-10.00
-10.00	33.30 ▲	-10.00

Avaliação de Políticas

- Como calculamos os V's para uma política fixa π ? ▪ Ideia 1: Transformar equações recursivas de Bellman em atualizações (como iteração de valor)

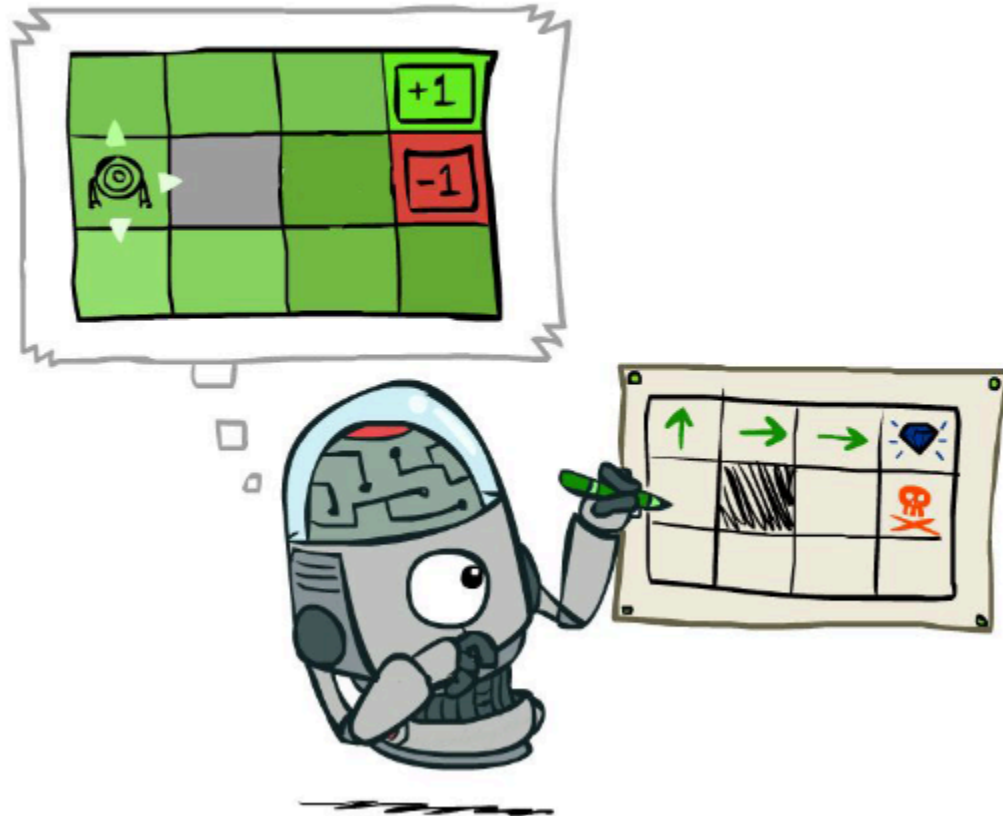
$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$



- Eficiência: $O(S^2)$ por iteração ▪ Ideia 2: Sem os máximos, as equações de Bellman são apenas um sistema linear Resolva com seu solucionador de sistemas lineares favorito

Extração de Políticas



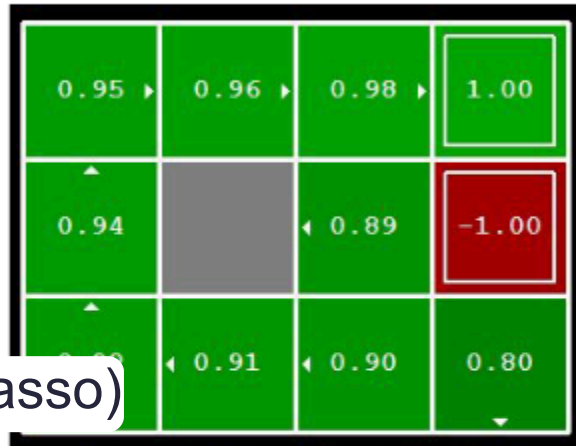
Calculando Ações a partir de Valores

- Vamos supor que temos os valores ótimos

$V^*(s)$. ▪ Como devemos agir?

- Não é óbvio!

- Precisamos fazer um mini-expectimax (um passo)



$$\pi^*(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

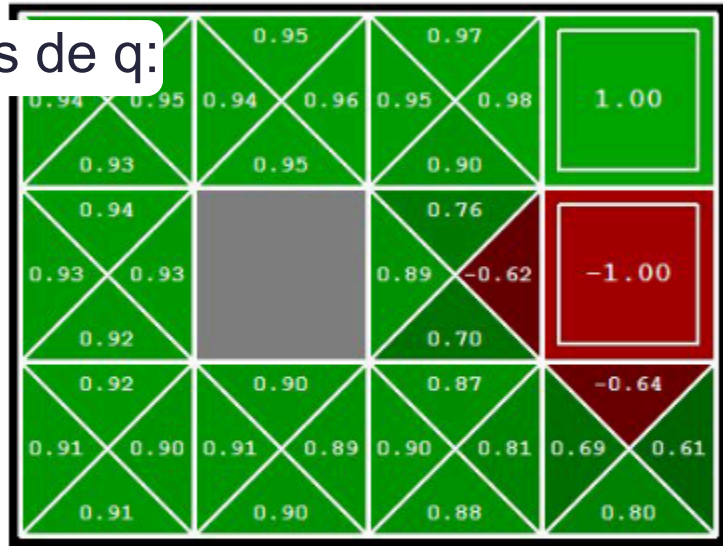
- Isso é chamado de extração de política, pois obtém a política implícita pelos valores

Cálculo de Ações a partir de Q-Values

- Vamos supor que temos os valores ótimos de q:

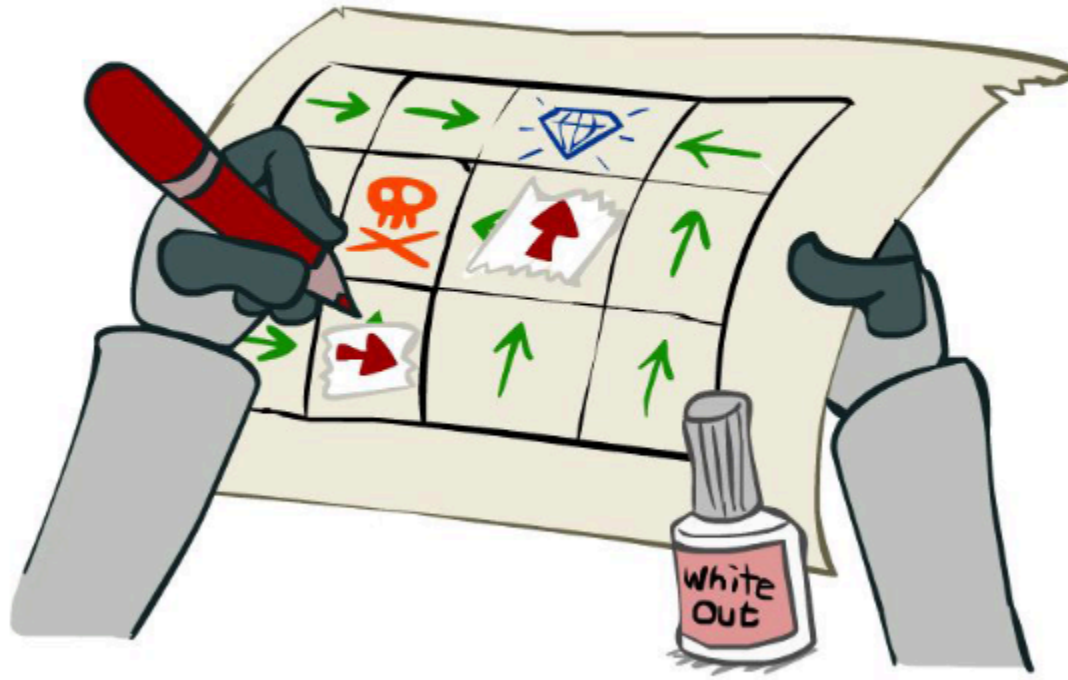
- Como devemos agir?
 - Totalmente trivial de decidir!

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$



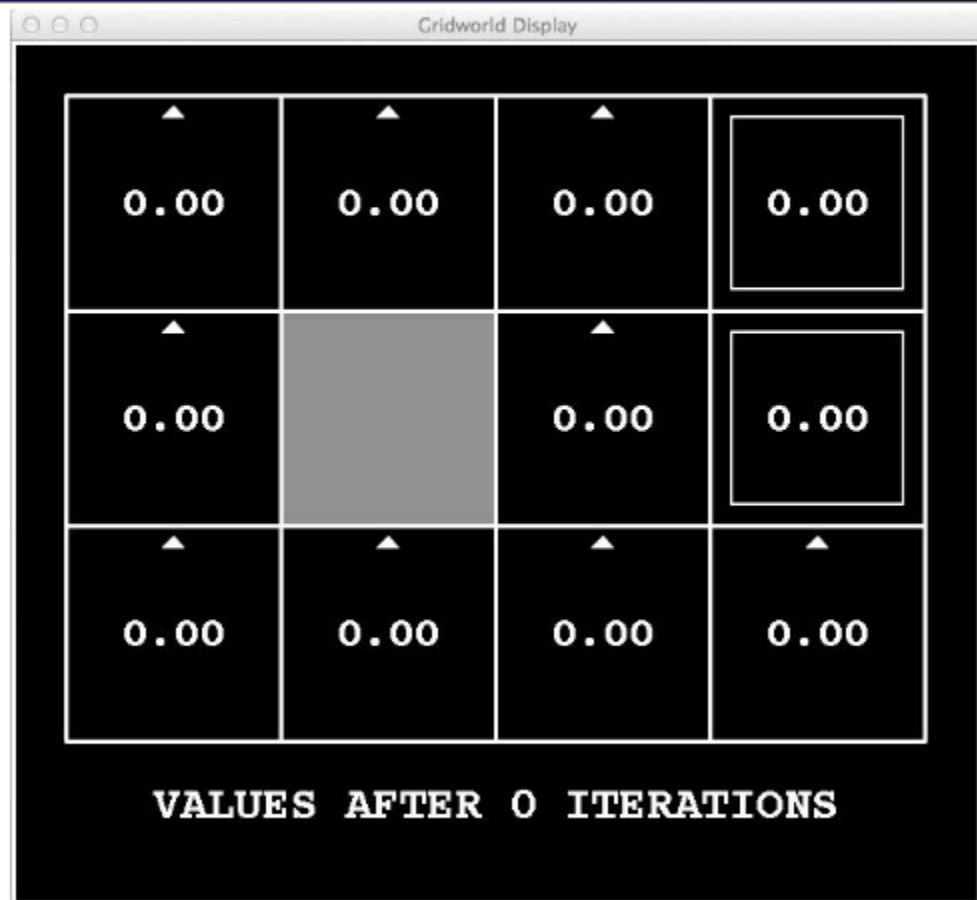
- Lição importante: ações são mais fáceis de selecionar entre valores q do que valores!

Iteração de Políticas



Problemas com a iteração do Value: demo

k=0



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=1



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=2



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=3



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=4



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=5



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=6



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=7



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=8



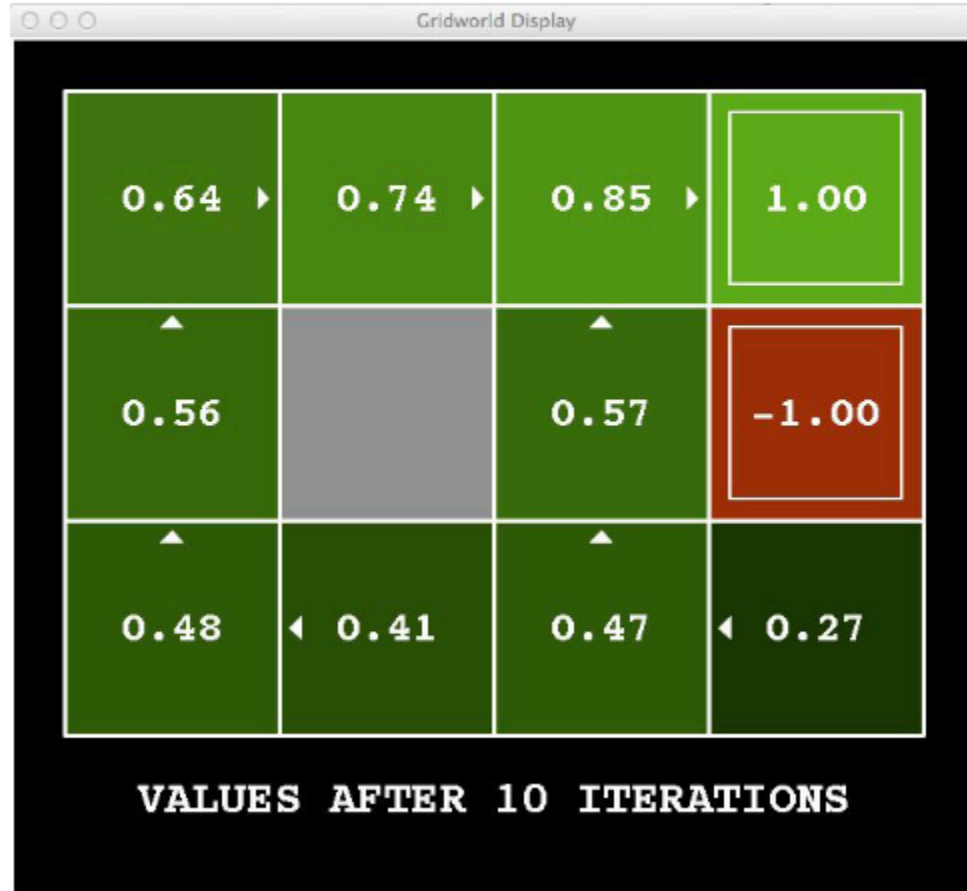
Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=9



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=10



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=11



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=12



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=100



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

Problemas com a Iteração de Valores

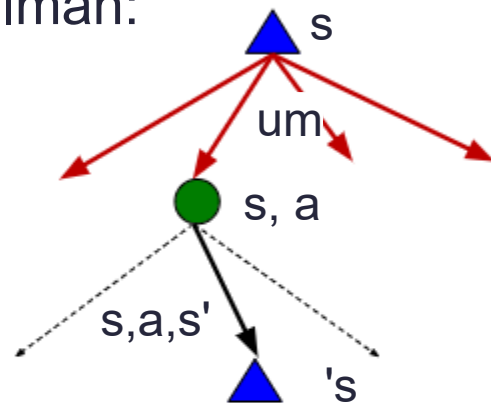
- A iteração de valor repete as atualizações do Bellman:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Problema 1: Está lento – $O(S^2A)$ por iteração

- Problema 2: O "máximo" em cada estado raramente muda

- Problema 3: A política frequentemente converge muito antes dos valores



Iteração de Políticas

- Abordagem alternativa para valores ótimos:
 - Passo 1: Avaliação de políticas: calcular utilidades para alguma política fixa (não ótima Utilidades!) até a convergência
 - Passo 2: Melhoria da política: atualizar a política usando uma antevisão em uma etapa com resultado utilidades convergentes (mas não ótimas!) como valores futuros
 - Repetir passos até que a política converga
- Isso é uma iteração de políticas
 - Ainda é o ideal!
 - Pode convergir (muito) mais rápido em algumas condições

Iteração de Políticas

- Avaliação: Para π de política fixa atual, encontre valores com avaliação de política:
 - Itere até que os valores convergam (pode rodar isso por muitas iterações):

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

- Melhoria: Para valores fixos, obtenha uma apólice melhor usando a extração de apólices
 - Visão antecipada de um passo (execute apenas uma vez):

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

Comparação

- Tanto a iteração de valores quanto a iteração de políticas calculam a mesma coisa (todos os valores ótimos).
 - Na iteração de valores:
 - Cada iteração atualiza tanto os valores quanto (implicitamente) a política. Não acompanhamos a política, mas tomar o máximo sobre as ações implicitamente a recalcula
 - Na iteração de políticas:
 - Fazemos várias passagens que atualizam utilitários com política fixa (cada passagem é rápida porque nós considere apenas uma ação, não todas)
 - Depois que a política é avaliada, uma nova política é escolhida (lenta como uma passagem de iteração de valor).
 - A nova política será melhor (ou acabou)
- Ambos são programas dinâmicos para resolver MDPs

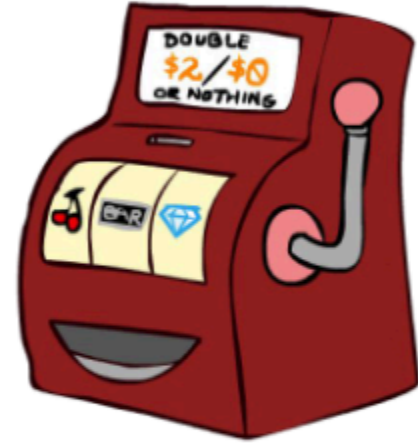
Resumo: Algoritmos MDP

- Então você quer....
 - Calcular valores ótimos: iteração de valor de uso ou iteração▪ de política
 - Calcular valores para uma política específica: usar avaliação▪ de políticas
 - Transforme seus valores em uma política: use extração de políticas (lookahead em um passo)
- Todos parecem iguais!
 - Basicamente, são – todas variações das atualizações▪ do Bellman.
 - Todas usam fragmentos▪ expectimax de visão em um passo.
 - Diferem apenas entre inserir uma política fixa ou maximizar as ações

Bandidos Duplos



\$1 toda vez

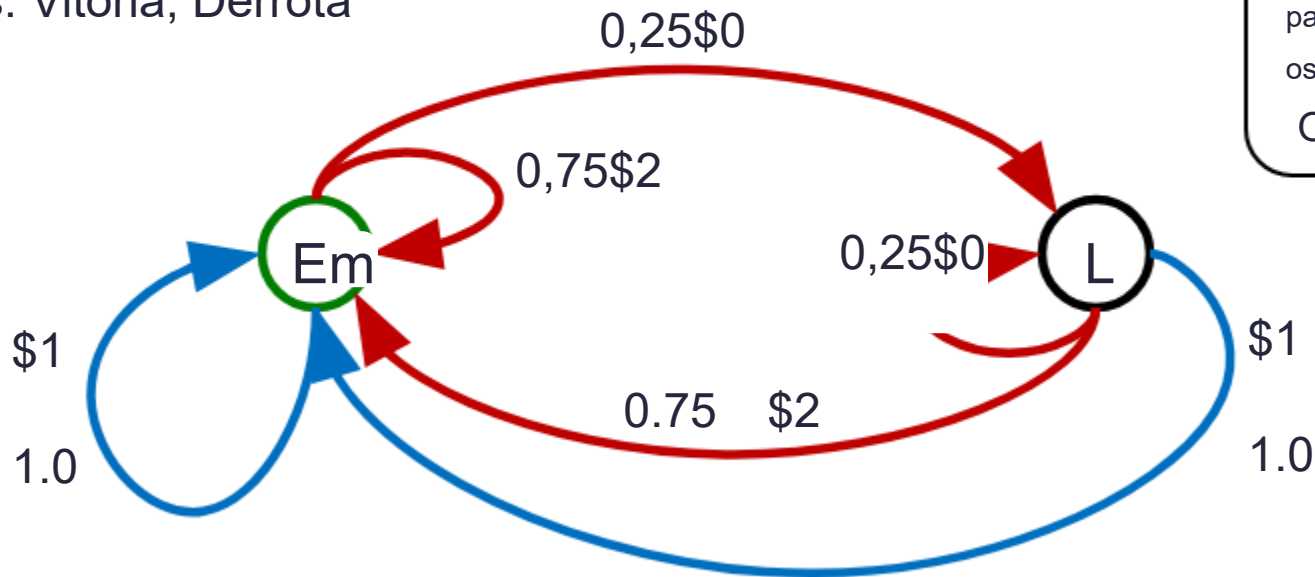


75% das vezes:
\$225% das vezes: \$0

Double-Bandit MDP

▪ Ações: Azul, Vermelho

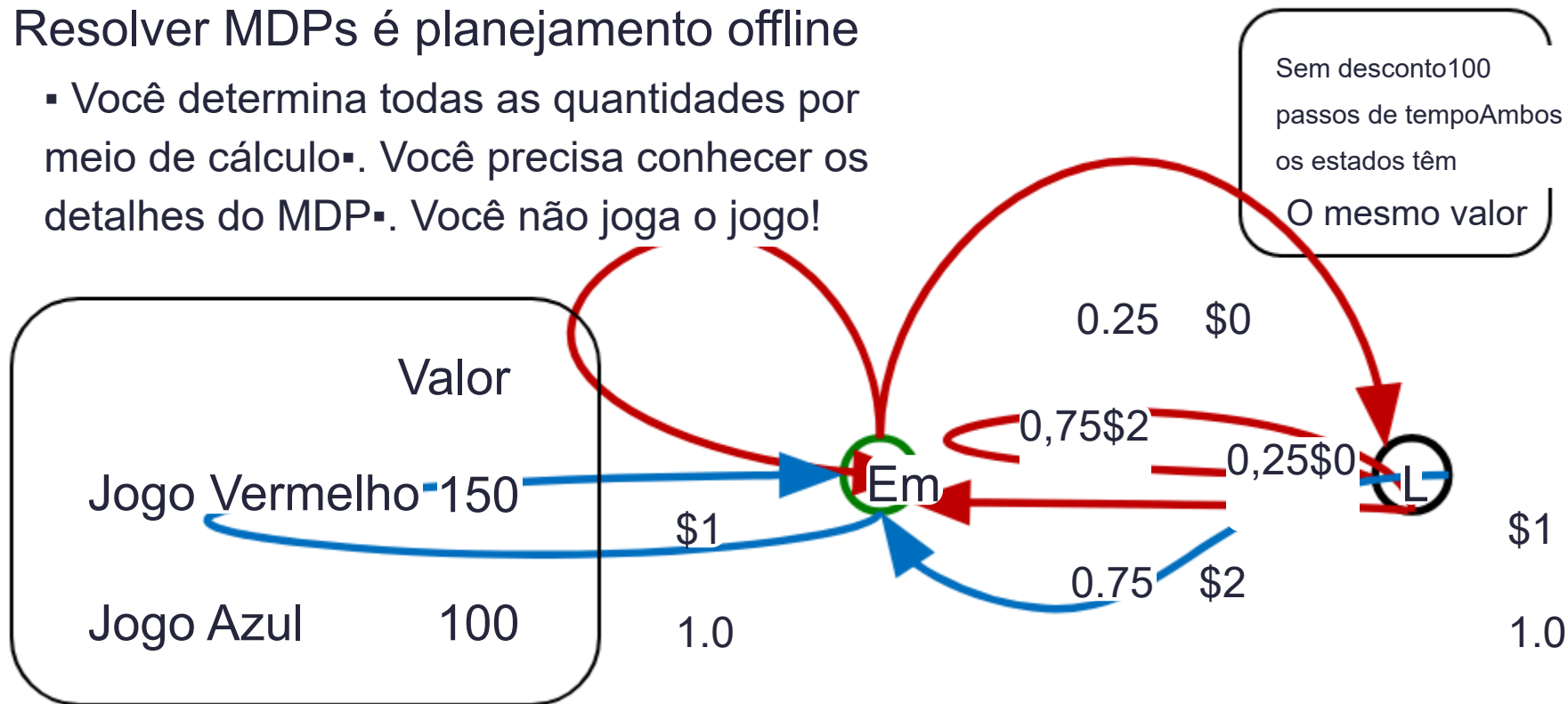
Estados: Vitória, Derrota



Sem desconto 100
passos de tempo Ambos
os estados têm
O mesmo valor

Planejamento Offline

- Resolver MDPs é planejamento offline
 - Você determina todas as quantidades por meio de cálculo.
 - Você precisa conhecer os detalhes do MDP
 - Você não joga o jogo!



Vamos jogar!



\$2 \$2 \$0 \$2 \$2\$2
\$2 \$0 \$0 \$0

Jogo mais difícil



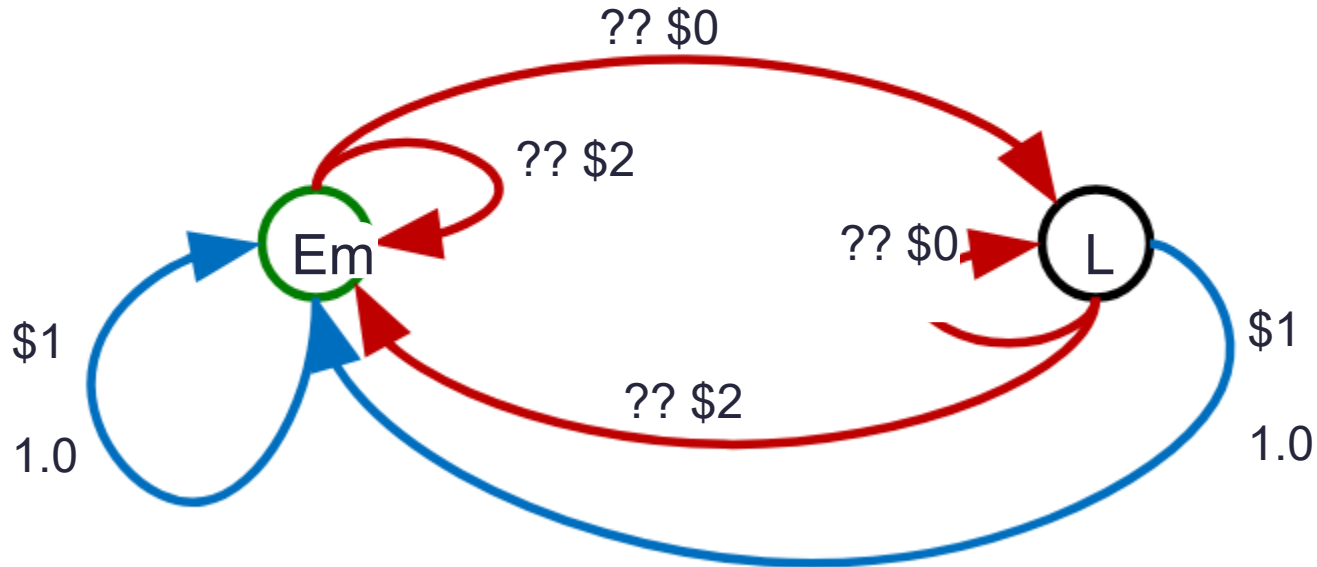
\$1 toda vez



?? % das vezes:
\$2?? % do tempo: \$0

Planejamento Online

- Regras mudaram! A chance de vitória do Red é diferente.



Vamos jogar!



\$0 \$0 \$0 \$2 \$0\$2
\$0 \$0 \$0 \$0

O que acabou de acontecer?

- Isso não era planejamento, era aprendizado!
 - Especificamente, aprendizado por reforço. Existia um MDP, mas não dava para resolver só com computação. Era preciso agir de verdade para descobrir
- Ideias importantes em aprendizado por reforço que surgiram
 - Exploração: você precisa tentar ações desconhecidas para obter informações
 - Exploração: eventualmente, você precisa usar o que sabe
 - Arrependimento: mesmo que aprenda inteligentemente, você comete erros
 - Amostragem: por acaso, você tem que tentar repetidamente
 - Dificuldade: aprender pode ser muito mais difícil do que resolver um MDP conhecido



Na próxima vez: Aprendizado por Reforço!
